

Hier sind 4 Prüfungsfragen passend zu Ihrem Beispiel, im Stil schriftlicher FIAE-Abfragen:

Frage 1 – Abstrakte Klasse und Vererbung

Erläutern Sie am Beispiel der Klassen `Client`, `Server` und `Workstation`:

1. Warum ist `Client` als **abstrakte Klasse** sinnvoll gewählt?
 2. Was bedeutet in diesem Zusammenhang die Aussage: „Ein Server *ist ein* Client“?
 3. Nennen Sie zwei Vorteile, die sich aus dieser Vererbungsstruktur für die Wartbarkeit oder Erweiterbarkeit des Codes ergeben.
-

Frage 2 – Abstrakte Methode und Polymorphie

Betrachten Sie die abstrakte Methode

```
public abstract double berechneLeistungsaufnahme(int betriebsstundenProTag);
```

1. Erklären Sie, was eine **abstrakte Methode** ist und warum sie nicht in der Klasse `Client` implementiert wird.
 2. Beschreiben Sie, wie das Prinzip der **Polymorphie** im Beispiel mit dem Array `Client[] clients` genutzt wird.
 3. Was passiert zur Laufzeit konkret, wenn in der Schleife über `clients` die Methode `berechneLeistungsaufnahme(...)` aufgerufen wird?
-

Frage 3 – „ist eine“- vs. „hat eine“-Beziehung

Im Beispiel gibt es sowohl Vererbung als auch Komposition.

1. Erklären Sie den Unterschied zwischen einer **„ist eine“-Beziehung** und einer **„hat eine“-Beziehung**.
 2. Nennen Sie je **ein konkretes Klassenpaar** aus dem Beispiel für jede dieser beiden Beziehungsarten und begründen Sie kurz Ihre Wahl.
 3. Warum ist es fachlich **nicht sinnvoll**, `Wartungsvertrag` von `Client` erben zu lassen?
-

Frage 4 – Wartungsvertrag und teuerster Vertrag

Im Beispiel besitzt `Wartungsvertrag` u.a. die Attribute `startDatum`, `endDatum` und `monatlicheKosten`. In `ClientDemo` wird der teuerste aktive Wartungsvertrag ermittelt.

1. Beschreiben Sie in eigenen Worten den Algorithmus, mit dem in `ClientDemo` der teuerste aktive Wartungsvertrag gefunden wird.
2. Welche Bedingungen müssen erfüllt sein, damit ein Vertrag in die Auswahl des „teuersten aktiven Vertrags“ einbezogen wird?
3. Nennen Sie eine mögliche fachliche Erweiterung der Klasse `Wartungsvertrag`, die in einem realen Projekt sinnvoll sein könnte, und begründen Sie kurz.

Hier sind kompakte Musterlösungen zu den vier Fragen – so, wie man sie in einer schriftlichen Prüfung erwarten könnte.

Musterlösung zu Frage 1 – Abstrakte Klasse und Vererbung

1. Warum **Client** abstrakt?

- **Client** beschreibt nur einen *allgemeinen Netzwerkteilnehmer* mit Leistung, IP-Adresse und Wartungsvertrag, ist aber kein konkreter Gerätetyp (man spricht in der Praxis eher von Server, Router, Workstation, ThinClient usw.).
- Von **Client** sollen daher **keine direkten Objekte** erzeugt werden, sondern nur von den spezialisierten Unterklassen.
- Die abstrakte Klasse bündelt **gemeinsame Attribute und Methoden** (*leistung*, *ipAdresse*, *wartungsvertrag*, *einschalten()*, *pruefeWartungsvertrag()*), ohne eine konkrete Ausprägung darzustellen.

2. „Server ist ein Client“

- **Server extends Client** bedeutet: **Server erbt** alle Eigenschaften und Methoden von **Client**.
- Fachlich heißt das: Ein **Server** ist ein spezieller Typ von **Client** (Netzwerkteilnehmer) mit zusätzlichen Merkmalen (*anzahlCpus*) und eigener Implementierung von *berechneLeistungsaufnahme(...)*.
- Überall dort, wo ein **Client** erwartet wird (z.B. im **Client[]**-Array), kann auch ein **Server** eingesetzt werden.

3. Zwei Vorteile der Vererbungsstruktur

- **Code-Wiederverwendung:** Gemeinsame Logik (Validierung im Konstruktor, *einschalten()*, *pruefeWartungsvertrag()*) steht zentral in **Client** und muss nicht in jeder Unterklasse doppelt implementiert werden.
- **Erweiterbarkeit/Polymorphie:** Neue Typen wie **ThinClient** oder **VirtuellerDesktop** können leicht hinzugefügt werden, indem sie **Client** erweitern und nur die spezifischen Teile implementieren (z.B. eigene Berechnung der Leistungsaufnahme). Bestehender Code, der mit Typ **Client** arbeitet (z.B. Schleife über **Client[]**), muss nicht angepasst werden.

Musterlösung zu Frage 2 – Abstrakte Methode und Polymorphie

1. Was ist eine abstrakte Methode, warum nicht in **Client** implementiert?

- Eine **abstrakte Methode** hat nur eine Methodensignatur, aber **keine Implementierung** (kein Methodenkörper). Sie zwingt Unterklassen, eine konkrete Implementierung bereitzustellen.
- In **Client** ist nicht klar, *wie* die Leistungsaufnahme berechnet werden soll, weil sich das Verhalten bei **Server**, **Router**, **Workstation**, **VirtuellerDesktop**, **ThinClient** fachlich unterscheidet.
- Deshalb wird in **Client** nur festgelegt: „Jeder Client **muss** eine Methode *berechneLeistungsaufnahme(int)* haben“, die konkrete Berechnung ist Aufgabe der Unterklassen.

2. Polymorphie im **Client[] clients**

- Im Array `Client[]` werden verschiedene konkrete Objekte gespeichert (`Server`, `Router`, `Workstation`, `VirtuellerDesktop`, ggf. `ThinClient`).
- Der Code in der Schleife behandelt alle Elemente **einheitlich** als `Client` und ruft z.B. `c.berechneLeistungsaufnahme(...)` auf.
- Welche Implementierung tatsächlich ausgeführt wird, hängt vom *konkreten Objekttyp* ab – das ist genau **Polymorphie** (ein Interface/Methodenname, verschiedene Implementierungen).

3. Was passiert zur Laufzeit beim Aufruf in der Schleife?

- Zur Laufzeit ermittelt die JVM den **dynamischen Typ** des Objekts, auf das `c` zeigt (z.B. `Server` oder `Router`).
- Dann wird die **jeweilige Überschreibung** von `berechneLeistungsaufnahme(...)` in dieser Unterklasse ausgeführt (z.B. die mit 90%-Faktor beim `Server` oder 0,6 beim `Router`).
- Dadurch ergibt sich je nach konkrete Klasse ein anderes Verhalten, obwohl im Code nur mit dem Typ `Client` gearbeitet wird.

Musterlösung zu Frage 3 – „ist eine“- vs. „hat eine“-Beziehung

1. Unterschied „ist eine“ vs. „hat eine“

- **„ist eine“-Beziehung** beschreibt eine **Vererbungsbeziehung**: Eine spezialisierte Klasse **ist** eine Ausprägung der allgemeineren Klasse (Subtyp). Beispiel: „Server ist ein Client“.
- **„hat eine“-Beziehung** beschreibt eine **Kompositions- oder Aggregationsbeziehung**: Ein Objekt **enthält** oder **nutzt** ein anderes Objekt als Bestandteil. Beispiel: „Client hat einen Wartungsvertrag“.

2. Je ein Klassenpaar aus dem Beispiel

- „ist eine“: `Server` und `Client` → `Server extends Client`. Ein `Server` ist ein spezieller `Client`, der alle Merkmale eines Clients besitzt, aber zusätzliche Eigenschaften (`anzahlCpus`) und eigenes Verhalten hat.
- „hat eine“: `Client` und `Wartungsvertrag` → Im `Client` gibt es ein Attribut `wartungsvertrag`. Ein `Client` **hat** (optional) einen `Wartungsvertrag`, der Informationen über Laufzeit und Kosten enthält.

3. Warum `Wartungsvertrag` nicht von `Client` erben lassen?

- Fachlich wäre die Aussage „Ein Wartungsvertrag ist ein Client“ unsinnig, da ein Vertrag kein Netzwerkteilnehmer ist.
- Ein `Wartungsvertrag` **besitzt keine Merkmale** eines Clients (keine IP-Adresse, keine Leistungsaufnahme), sondern beschreibt nur eine **Dienstleistungsvereinbarung** zu einem Client.
- Eine Vererbungsbeziehung würde hier das Modell verfälschen – richtig ist eine Komposition: Ein `Client` hat einen `Wartungsvertrag`.

Musterlösung zu Frage 4 – Wartungsvertrag und teuerster Vertrag

1. Algorithmus zum Finden des teuersten aktiven Wartungsvertrags In Worten, basierend auf `ClientDemo`:

- Es wird über alle Elemente im Array `clients` iteriert.

- Für jeden **Client** wird dessen **Wartungsvertrag** mit `getWartungsvertrag()` geholt.
- Ist kein Vertrag vorhanden (**null**) oder der Vertrag nicht aktiv (`!istAktiv()`), wird er übersprungen.
- Andernfalls wird geprüft, ob dieser Vertrag **teurer** ist als der bisher als teuerster gemerkte Vertrag (Vergleich von `getMonatlicheKosten()`).
- Wenn ja, wird die Referenz auf diesen Vertrag als **teuersterAktiverVertrag** gespeichert.
- Nach der Schleife wird der teuerste gefundene aktive Vertrag (falls vorhanden) ausgegeben.

2. Bedingungen, um in die Auswahl einzugehen

- Der Vertrag darf nicht **null** sein (es muss also tatsächlich ein Wartungsvertrag für den Client existieren).
- Die Methode `istAktiv()` muss **true** liefern, d.h. das aktuelle Datum liegt zwischen **startDatum** und **endDatum** (einschließlich).
- Nur Verträge, die diese beiden Bedingungen erfüllen, können als teuerster aktiver Vertrag kandidieren.

3. Mögliche fachliche Erweiterung von **Wartungsvertrag** (Beispiele, eins genügt in der Prüfung, mit Begründung)

- Ergänzung um ein Attribut **serviceLevel** (z.B. „Gold“, „Silber“, „Bronze“) und/oder Reaktionszeiten im Fehlerfall (z.B. **maxReaktionszeitInStunden**). → Begründung: In echten Wartungsverträgen sind Service-Level-Agreements (SLA) üblich, die genau solche Qualitätszusagen abbilden.
- Ergänzung um **kuendigungsfristInTagen** und eine Methode wie `berechneKuenigungsdatum()`. → Begründung: Kunden müssen wissen, bis wann sie einen Vertrag ordentlich kündigen können.
- Ergänzung um eine Methode **double berechneGesamtkosten()**, die auf Basis von Laufzeit und monatlichen Kosten die Gesamtkosten des Vertrags ermittelt. → Begründung: Für kaufmännische Auswertungen sind Gesamtkosten wichtig.

Diese Beispiele zeigen, dass **Wartungsvertrag** als eigene Klasse gut erweiterbar ist und fachliche Anforderungen aus der Praxis abbildet.

Hier ein kompaktes UML-Klassendiagramm als ASCII, passend zu deinem Beispiel (inkl. ThinClient). Du kannst es direkt ins Skript übernehmen oder an die Tafel bringen.

```
+-----+
| <<abstract>> |
| Client |
+-----+
| - leistung: double |
| - ipAdresse: String |
| - wartungsvertrag: |
| Wartungsvertrag |
+-----+
| + Client(leistung, |
| ipAdresse, |
| wartungsvertrag) |
| + getLeistung(): double |
| + getIpAdresse(): String |
| + getWartungsvertrag(): |
| Wartungsvertrag |
```

```

| + einschalten(): void |
| + pruefeWartungsvertrag() |
| : void |
| + berechneLeistungs- |
| aufnahme( |
| betriebsstunden: int|
| ): double {abstract}|
+-----+
^ ^ ^ ^ ^
| | | | |
+-----+-----+-----+-----+-----+
| | | | |
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+
-----+ +-----+
| Server | | Router | | Workstation | | VirtuellerDesktop | | ThinClient |
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+
-----+ +-----+
| - anzahlCpus: int | | - anzahlPorts: int | | - hatGrafikkarte: | | -
providerName: String | | - bandbreite: |
| | | | boolean | | | double |
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+
-----+ +-----+
| + Server(leistung, | | + Router(leistung, | | + Workstation( | | +
VirtuellerDesktop( | | + ThinClient( |
| ipAdresse, | | ipAdresse, | | leistung, | | leistung, | | leistung, |
| vertrag, | | vertrag, | | ipAdresse, | | ipAdresse, | | ipAdresse, |
| anzahlCpus) | | anzahlPorts) | | vertrag, | | vertrag, | | vertrag, |
| + getAnzahlCpus(): int | | + getAnzahlPorts(): | | hatGrafik- | | providerName)
| | bandbreite) |
| + berechneLeistungs- | | int | | karte) | | + getProviderName(): | | +
getBandbreite(): |
| aufnahme(int): double | | + berechneLeistungs- | | + isHatGrafik- | | String | |
double |
| | | aufnahme(int): | | karte(): bool | | + berechneLeistungs- | | +
berechneLeis- |
| | | double | | + berechneLeis- | | aufnahme(int): double | | tungsaufnahme( |
| | | | tungsaufnahme( | | | | int): double |
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+
-----+ +-----+
+-----+

+-----+
| Wartungsvertrag |
+-----+
| - vertragsNummer: String |
| - startDatum: LocalDate |
| - endDatum: LocalDate |
| - monatlicheKosten: double |
+-----+
| + Wartungsvertrag(vertragsNummer, |
| startDatum, endDatum, |
| monatlicheKosten) |
| + getVertragsNummer(): String |

```

```
| + getMonatlicheKosten(): double |  
| + istAktiv(): boolean |  
+-----+
```

```
(Client) 1 --- 0..1 (Wartungsvertrag)  
"Client hat optional einen Wartungsvertrag"
```